

PolyDrive

Low-poly racing game with Arduino joystick support

Usage guide, configuration reference and debugging notes

1. Overview

PolyDrive (game_3d.py) is a single-player time-trial racing game built with the Ursina engine (Python 3D, built on Panda3D). You drive a low-poly car along a flat road that has long bends, an S-bend and two U-turns, pass three checkpoints, collect coins for speed boosts and cross the finish line as fast as you can. It can be played on the keyboard, with an Arduino joystick board connected over USB serial, or both at the same time.

- Course: flat road, about 2,300 units long, 16 units wide, with barriers on both sides.
- Race: timer starts when the car first moves; three checkpoints, the third is the finish line.
- Coins: about 35 gold discs along the road, each gives a speed boost.
- Sound: engine, tyre squeal, barrier thump and scrape, checkpoint, coin and win jingles, and a looping music track. All generated by the game itself.
- Controls: keyboard and/or two Arduino joysticks (analog throttle, brake and steering).

2. Requirements and installation

Item	Version / note
Operating system	Windows 11 (developed and tested there); Ursina also runs on macOS and Linux
Python	3.11
ursina	7.0.0 (pinned - see Troubleshooting: colors changed between versions)
pyserial	3.5 (only needed for the Arduino joystick; the game runs without it)
Graphics	Any GPU able to run Panda3D / OpenGL

First-time setup

```
cd wireless-game-controller
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements_stage2.txt
```

requirements_stage2.txt contains pygame (used by the 2D game), ursina and pyserial.

3. Running the game

```
.venv\Scripts\python game_3d.py
```

- The first start generates the sound files into the sounds/ folder (a few seconds). Later starts reuse them.
- Startup builds roughly 2,000 objects, so the window can take several seconds to appear.
- With the Arduino connected, the top-right corner shows 'Joystick: connected on COMx'.
- Esc quits the game.

4. Controls

Keyboard

Key	Action
W / Up arrow	Accelerate (the longer you hold, the harder it pushes)
S / Down arrow	Brake, then reverse

Key	Action
A, D / Left, Right arrows	Steer (eases in and re-centres smoothly)
Space	Drift (tighter turning)
R or Enter	Return to the last checkpoint (timer keeps running)
T or Backspace	Start over from the beginning (coins come back)
M	Mute / unmute the music
Esc	Quit

Arduino joystick

Your board sends two sticks. Both work the same way; whichever is pushed furthest wins.

Stick movement	Action
Left / right	Steer (analog: a small push turns a little)
Up	Accelerate (analog: how far you push = how hard)
Down	Brake / reverse (analog)
Press the stick (either)	Drift

Note: Leave both sticks untouched while the game starts. It measures their resting position for about half a second and treats that as the centre.

5. Game rules

- Timer: starts when the car first moves. It shows Current, your Record (best finish) and the Difference (red = slower, green = faster).
- Checkpoints: 3 in total, at one third, two thirds and the end of the course. The counter (bottom left) shows how many you have passed. R / Enter returns you to the last one.
- Finish: passing the third checkpoint stops the timer and shows the win screen. R / Enter or T / Backspace starts a new race.
- Barriers: the car is kept on the road. Rubbing a barrier slows the car and plays a scrape; a hard hit plays a thump.
- Coins: each coin gives an instant +5 speed, plus 2.5 seconds of boost (up to 6 seconds stacked). While boosting, the car gets extra acceleration and a top speed 35 percent higher. The HUD shows BOOST and the seconds left.
- Falling off the world returns you to the last checkpoint.
- HUD speed: full speed reads 196 km/h. The physics top speed is 60 units per second; the number shown is scaled (see TOP_SPEED_DISPLAY).

6. Arduino joystick hardware

How it connects

The board appears as a serial (COM) port. The game reads lines of text from it on a background thread and never blocks the game loop. If the board is unplugged the game keeps running on the keyboard and retries every 2 seconds.

Data format (verified on the project board)

```
745,444,1,506,514,1■
^ stick 1: x, y, button   ^ stick 2: x, y, button

Baud rate : 115200
Axes      : 0-1023, centre near 506-514
Button    : 1 at rest, 0 when pressed (active low)
```

The parser accepts any text line containing numbers: the 1st, 2nd and 3rd numbers are stick 1 (x, y, button) and the 4th, 5th and 6th are stick 2. A board that sends only three numbers works as a single stick.

Calibration

- The first 30 lines received are averaged to find each stick's centre.
- JOY_RANGE (400) is how far the raw value moves from the centre at full push. The range widens automatically if a stick reads further.
- JOY_DEADZONE (0.12) ignores tiny movements around the centre.

Finding out what a board sends

```
.venv\Scripts\python tools\serial_check.py
```

The tool lists COM ports, then reads the chosen port at 9600, 115200, 57600, 38400 and 19200 baud. The baud rate that prints readable lines is the right one. Close the Arduino IDE Serial Monitor first: only one program can use a COM port at a time.

If you use a blank Arduino instead

arduino/joystick_controller/joystick_controller.ino is a ready sketch (VRx to A0, VRy to A1, SW to D2, 5V and GND). It sends 'x,y,button' at 115200 baud, which the game understands as a single stick. Because the sketch sends 1 when the button is pressed, set JOY_BUTTON_ACTIVE_LOW = False when using it.

7. Configuration reference

All tuning values are constants near the top of each section in game_3d.py. Change them and restart the game.

Driving

Constant	Default	Meaning
MAX_SPEED	60	Top speed in world units per second
TOP_SPEED_DISPLAY	196	Number shown on the HUD at top speed
REVERSE_SPEED	-15	Top reverse speed
ACCEL / BRAKE_DECEL / FRICTION	30 / 50 / 10	Acceleration, braking and coasting slow-down
PEDAL_PRESS / PEDAL_RELEASE	1.4 / 3.0	How fast a key-driven pedal fills / empties (per second)
TURN_SPEED	110	Turn rate in degrees per second at full steering
STEER_RESPONSE / STEER_RETURN	4.0 / 6.0	How quickly the wheel turns toward the key / re-centres
HIGH_SPEED_STEER	0.6	Share of steering lock left at top speed
BODY_LEAN	6	Visual body tilt in degrees
BARRIER_SCRAPE	2.5	Speed lost per second while rubbing a barrier

Course, camera and coins

Constant	Default	Meaning
ROAD_WIDTH	16	Road width
COURSE	list	Sequence of ('S', length) straights and ('A', degrees, radius) arcs; + = right, - = left
CAMERA_BACK / CAMERA_HEIGHT	12 / 6.5	Chase camera distance and height
CAMERA_LOOK_AHEAD / camera.fov	8 / 80	Aim point ahead of the car; field of view
COIN_SPACING	30	Samples between coins (1 sample = 2 units)
BOOST_SECONDS / BOOST_MAX_SECONDS	2.5 / 6	Boost per coin and the stacking limit
BOOST_TOP_FACTOR / BOOST_ACCEL	1.35 / 30	Top-speed multiplier and extra acceleration while boosting
COIN_KICK / COIN_PICKUP_RADIUS	5 / 2.6	Instant speed gain and pickup distance
MUSIC_VOLUME	0.35	Music loudness (0 to 1)

Joystick

Constant	Default	Meaning
SERIAL_PORT	None	Fixed port such as 'COM3'; None = auto-detect
SERIAL_BAUD	115200	Must match the Arduino program
JOY_RANGE	400	Raw distance from centre at full push
JOY_DEADZONE	0.12	Dead area around the centre
JOY_INVERT_X / JOY_INVERT_Y	False / True	Flip an axis (Y True means stick up = accelerate)
JOY_BUTTON_ACTIVE_LOW	True	True if the button reads 0 when pressed

8. How it works (architecture)

Files

Path	Purpose
game_3d.py	The whole game (one file, organised in labelled sections)
game_2d.py	Separate earlier 2D coin game (not covered here)
tools/serial_check.py	Shows what an Arduino sends, per baud rate
tools/build_docs.py	Regenerates this documentation (Markdown and PDF)
arduino/joystick_controller/	Sketch for a blank Arduino + joystick module
sounds/	Generated audio (git-ignored, recreated on first run)
docs/	This documentation
requirements_stage2.txt	Pinned dependencies

Sections of game_3d.py (in order)

- 1 Imports and app: Ursina(development_mode=False), asset folder set to sounds/.
- 2 Lighting and sky: directional sun, ambient light, unlit sky dome.

- 3 Course: COURSE turns into 'samples', one point every 2 units along the road centre-line, each with (x, z, heading). Everything else is built from those samples.
- 4 Ground, trees and mountains: a large ground slab; trees and mountains are placed only where distance_to_road() is large enough.
- 5 Road tiles and barriers: solid 2-unit-thick slabs (long on straights, short on curves) with barriers and a dashed centre line.
- 6 Start, checkpoint and finish gates.
- 7 Coins: gold discs placed on the road, weaving left, centre and right.
- 8 Car: built from cubes; physics constants and state variables.
- 9 Sound: file generation, load_sound() helper with a silent fallback, and the render_music() track.
- 10 Helper functions: approach(), get_ground_y(), find_track_index(), place_car(), reset_race(), return_to_checkpoint().
- 11 HUD: dark-blue panels built on camera.ui.
- 12 Camera settings and the Joystick class (background thread).
- 13 input(key) and update(): the per-frame game loop.

Per-frame loop (update)

- 1 Read pedals: keyboard keys ramp the pedal up; joystick gives an analog value.
- 2 Apply acceleration, braking, friction and the coin boost; enforce the speed limit.
- 3 Steering: ease the wheel toward the input, turn the car by $\text{steer} * \text{TURN_SPEED} * \text{grip}$.
- 4 Move the car forward, find the nearest road sample, and push the car back inside the barriers (with scrape and thump).
- 5 Vertical physics: a downward ray finds the road or ground height; gravity applies if the car is airborne.
- 6 Timer, checkpoints and finish detection (based on the nearest road sample index).
- 7 Coins: spin, and collect when the car is within 2.6 units.
- 8 Chase camera follows and looks ahead of the car (world up vector, so it never rolls).
- 9 Sound: engine pitch and volume, skid and scrape volumes.
- 10 HUD text is refreshed.

Audio

No audio files are shipped. make_sounds() writes 16-bit mono WAV files into sounds/ the first time the game runs: engine loop (harmonics of 55 Hz), tyre squeal (noise plus tones), scrape, thump, checkpoint and win jingles, coin chime and an 8-bar music loop at 124 BPM (kick, snare, hi-hat, bass and arpeggio over Am - F - C - G, with a drum break in the last bar). Engine pitch follows speed. If sounds cannot be created or loaded, SilentSound stand-ins are used and the game runs without audio.

Joystick thread

Joystick._run() runs as a daemon thread: find the port, open it, read lines, parse numbers, calibrate, then write x, y and button into the joystick object. The game loop only reads joystick.x, joystick.y and joystick.button. On any serial error the values reset to zero and the thread retries.

9. Verification performed

The game was checked with scripted tests (no human driving) before this documentation was written:

Check	Result
Code compiles, static check (pyflakes)	Clean (only Ursina star-import notices)
Production start (dev mode off, no editor overlay)	Confirmed development_mode is False
Road solidity: ray cast at 78 points along the whole course	All found the road at height 0.15
Autopilot lap: full throttle, steering toward the road	Finished, 3 of 3 checkpoints, 11 coins, max speed 81 (60 x 1.35 cap)
Reset and return-to-checkpoint after a finish	Worked
Fake two-stick Arduino stream	Second stick alone gave full steering and throttle; active-low button detected
pyserial missing	Game starts, status reads 'install pyserial to use it'
Fresh clone: sounds/ deleted	All 8 sound files regenerated and loaded

Note: Not verified by these tests: how the game looks and sounds, and behaviour with the real Arduino hardware. Do a manual play-through (checklist below) on the target machine.

Manual check before a release

- Game opens with the scene visible (not white) and the HUD panels drawn.
- Car accelerates, steers and brakes on the keyboard; engine pitch rises with speed.
- Tyre squeal on hard braking; thump and scrape against a barrier.
- Coins disappear when collected, BOOST timer appears, speed goes above 196.
- All 3 checkpoints count; R / Enter returns to the last; T / Backspace restarts and restores coins.
- Finish shows the win screen and Record / Difference update on the next lap.
- Joystick: status says connected; up = accelerate, right = steer right, button = drift, for both sticks.
- Music plays and M mutes it; Esc quits.

10. Troubleshooting and debugging

Symptom	Cause and fix
Whole 3D view is white, HUD text visible	Ursina 7 changed color.rgb() to take 0-1 values. Use color.rgb32() / rgba32() for 0-255 values. Keep ursina pinned to 7.0.0.
Game hangs at startup, no window	Something in the object-building code loops forever or is very slow. Use python -X faulthandler and faulthandler.dump_traceback_later(30) to see where it is. (Past cause: a while loop that never incremented its counter.)
Startup is slow	Ursina scans the asset folder recursively. The game sets the asset folder to sounds/ so it never scans .venv. Do not move that line.
Car sinks into the road / hills	raycast hit.point is local to the hit object. Use hit.world_point (already done in get_ground_y). Road colliders must be top-level entities, not children of a rotated parent.
Camera turns upside down or rolls	Entity.look_at uses the camera's own up vector by default. The game passes up=Vec3(0, 1, 0).
No sound	Check the sounds/ folder has 8 .wav files (delete it to regenerate). Console prints '[sound] ... unavailable' if a clip fails. Press M in case it is muted.

Symptom	Cause and fix
Status: 'install pyserial to use it'	pip install pyserial in the same .venv the game runs from.
Status: 'no Arduino found'	Check the USB cable and drivers; run tools/serial_check.py; set SERIAL_PORT = 'COM3' (your port).
Serial port opens but the joystick does nothing	Wrong baud rate (run serial_check.py) or the Arduino IDE Serial Monitor is still open and holding the port.
Up brakes / right steers left	Flip JOY_INVERT_Y / JOY_INVERT_X.
Car creeps or steers by itself	A stick was held during startup or has drift: raise JOY_DEADZONE, restart with sticks released.
Cannot reach full throttle on the stick	Lower JOY_RANGE (the raw value never travels as far as 512 on some sticks).
Button drifts constantly or never	Toggle JOY_BUTTON_ACTIVE_LOW.
Music too loud or quiet	Change MUSIC_VOLUME; the engine volume is set in the Sound part of update().

Useful debug techniques

- Scripted driving: run the game file with app.run() removed, set ursina.time.dt = 1/60, hold keys through ursina.held_keys, and call update() in a loop, then inspect speed, track_index and checkpoints_passed. This is how the lap test above was done.
- Fake Arduino: replace sys.modules['serial'] with an object whose readline() returns test lines to exercise the joystick code without hardware.
- Raw serial data: tools/serial_check.py prints exactly what the board sends.

11. Known limitations and ideas

- Single course; the course is data (COURSE list), so new layouts are easy to add. Avoid layouts where road parts come within about 30 units of each other.
- No opponents, ghost car or saved records: the Record resets when the game closes.
- The car does not collide with trees or mountains (they are scenery); the barriers keep it on the road.
- Joystick calibration assumes the sticks are at rest at startup.
- Only one serial device is used; the game picks the first likely Arduino / USB serial port.
- Ideas: save best time to a file, ghost replay, more courses, wireless controller support, gamepad support through pygame.

12. Change log of this development session

- Fixed the white screen (Ursina 7 color API), then removed the shadow shader and fog.
- Rebuilt the course: from a stadium loop with a jump, to a straight road, to the current flat course with bends and U-turns.
- Analog pedals and smoothed steering; body lean; chase camera that looks ahead.
- PolyTrack-style HUD with checkpoints, Record / Current / Difference.
- Generated sound effects and music; coins with speed boost.
- Arduino joystick support over serial, verified against the real board's output (two sticks, 115200 baud).
- Production hardening: dev mode off, asset folder fix, silent audio fallback, respawn at checkpoint, pinned requirements, .gitignore cleaned (sounds/ ignored).